

Uso Etico IA Taller 1 IA

Función #1

uniformCostSearch en algorithms/search.py (Punto 3)

Explicación:

Antes de escribir código, ya sabía que UCS es conceptualmente el mismo algoritmo de Dijkstra que implementé en el reto 4 de EDA (con mis propias estructuras de mapa, priority queue y pila para reconstrucción de camino). Sin embargo, noté que este proyecto ya trae sus propias abstracciones en algorithms/utils.py (Stack, Queue, PriorityQueue) que reemplazan los TADs que yo mismo construí en EDA. No tenía claro cuáles funciones de esas estructuras usar, ni cómo se conectaban con el resto del framework (SearchProblem, getSuccessors), porque nunca antes había trabajado con esta interfaz específica del curso.

Justificación:

Por eso recurrí a Claude, pidiéndole primero que me explicara cada función usada por mis compañeros en depthFirstSearch y breadthFirstSearch (ya implementadas), para entender qué hacía cada una antes de tocar mi propia función. Le pedí explícitamente una tabla comparativa entre las estructuras que yo mismo construí en EDA (mp, pq, al, st de mi Dijkstra) y sus equivalentes en este proyecto, además de pseudocódigo genérico (no código Python de la función), para no recibir la solución directamente. Claude respondió, entre otras cosas: "pq.insert(pq, prioridad, vertice) → utils.PriorityQueue().push(item, priority) — ojo: orden de argumentos invertido... No existe grafo precomputado: llamas problem.getSuccessors(state), que te devuelve directo una lista de tuplas (vecino, accion, costo)... En vez de guardar predecesores y reconstruir con un stack (path_to), cada elemento que metes a la cola ya lleva pegada la lista de acciones acumuladas hasta ese estado." (Claude, Anthropic, 2026)

Aquí tienes la tabla comparativa entre tus estructuras de EDA (Dijkstra) y lo que efectivamente usan `depthFirstSearch` y `breadthFirstSearch` ya implementados en este repo — así ves de primera mano qué funciones/patrones existen antes de escribir tu propia versión.

Concepto	Tu Dijkstra en EDA	Cómo lo resuelve <code>depthFirstSearch</code> (search.py:32-51)	Cómo lo resuelve <code>breadthFirstSearch</code> (search.py:60-80)
Estructura de frontera	pq (priority queue propia)	<code>utils.Stack()</code> — LIFO	<code>utils.Queue()</code> — FIFO
Insertar en frontera	<code>pq.insert(pq, prioridad, vertice)</code>	<code>frontier.push((state, actions))</code> — sin prioridad, solo el par estado+camino	<code>frontier.push((state, actions))</code> — mismo par
Sacar de frontera	<code>pq.remove(pq) → vertice</code>	<code>frontier.pop()</code> → último insertado	<code>frontier.pop()</code> → pero como <code>Queue.push</code> inserta al inicio (<code>list.insert(0, item)</code>) y <code>pop()</code> saca del final, el efecto neto es FIFO real
Estructura de visitados	mp (mapa) con <code>{marked, edge_from, dist_to}</code> por vértice	<code>visited = set()</code> — solo un conjunto simple, sin info extra	<code>visited = set()</code> — igual
Cuándo marcas visitado	Al sacar de pq (<code>info_v["marked"] = True</code>)	Al expandir, después de sacar de la pila (línea 44-45): primero revisa meta, luego marca	Al insertar en la frontera (línea 66 y 75-76), no al expandir — evita duplicados en la cola
Verificar si ya visitado antes de reinsertar	<code>if not info_w['marked']:</code> antes de relajar	<code>if successor not in visited:</code> antes de hacer push	<code>if successor not in visited:</code> antes de hacer push
Reconstrucción del camino	<code>edge_from</code> guardado por vértice + función <code>path_to</code> que arma una pila al final	No usa <code>edge_from</code> . Cada item de la frontera ya lleva pegada la lista de acciones acumuladas: (<code>successor, actions + [action]</code>)	Igual: (<code>successor, newActions</code>) con <code>newActions = actions + [action]</code>
Condición de	No aplica igual —	<code>if problem.isGoalState(state):</code>	Igual, se revisa al sacar de la

Condición de parada / meta	No aplica igual — Dijkstra corre hasta vaciar la cola completa (calcula distancia a todos)	if problem.isGoalState(state): return actions — se revisa al sacar de la pila, antes de marcar visitado	Igual, se revisa al sacar de la cola
Costo acumulado	dist_to guardado en el mapa por vértice	No lo llevan — DFS no le importa el costo, solo encontrar cualquier camino	No lo llevan tampoco — BFS minimiza pasos, no costo, así que no necesita acumular costo real
Info del sucesor que usan	edg.weight(arco) desde un objeto arco explícito	Ignoran el tercer valor de la tupla: for successor, action, stepCost in problem.getSuccessors(state): — el stepCost ni se usa	Igual, stepCost se ignora

Recomendaciones que salen de esta tabla (para tu UCS)

1. El patrón (estado, acciones) que usan DFS/BFS es el que debes extender, agregando el costo acumulado como tercer elemento de la tupla — así te ahorras por completo el edge_from + path_to de tu EDA.
2. La diferencia crítica entre BFS y DFS que debes replicar bien en UCS: BFS marca visitado *al insertar* (evita duplicados en cola), DFS marca *al expandir*. Para UCS, lo correcto (como en Dijkstra) es marcar *al expandir/sacar de la cola* — porque solo ahí sabes con certeza que ese es el costo mínimo confirmado para ese estado (por eso tu propio código usa visitados.add(estadoActual) después del pop, igual que el patrón de DFS, no el de BFS).
3. stepCost es lo único que ni DFS ni BFS usan pero que tú sí necesitas — es el dato que hace que tu problema sea Dijkstra y no BFS con pesos ignorados.

Con esa guía escribí mi propia versión reusando la lógica de mi Dijkstra. Al terminar, corrí mi código y busque si había bugs, y tenía un error puntual que copilot me marco el cual fue: if estadoActual is problem.isGoalState(estadoActual): comparaba por identidad de objeto en vez de evaluar el booleano retornado, lo que impedía detectar la meta. También señaló que el utils.raiseNotDefined() residual al final debía ser return [], siguiendo el patrón ya usado en DFS/BFS. Apliqué ambas correcciones yo mismo y verifiqué el resultado corriendo uniformCostSearch contra los 10 mapas de layouts/diagnostic/, comparando el costo obtenido contra breadthFirstSearch en los mismos mapas: en todos los casos el costo de UCS fue igual o menor al de BFS (nunca mayor), lo cual es la evidencia esperada de que UCS respeta la optimalidad de costo.

Aprendizaje breve:

Confirmé que el "traducir" un algoritmo de una implementación con TADs propios a una librería/framework ajeno no es reescribir el algoritmo desde cero, sino identificar la equivalencia funcional entre estructuras (mi PriorityQueue de EDA vs. la de este proyecto, mi reconstrucción de camino con edge_from/pila vs. acumular acciones en cada nodo de la frontera). El bug que cometí (is en vez de evaluar el booleano) fue un error sintáctico básico, no conceptual, el diseño del algoritmo en sí lo tenía correcto desde el primer intento. Esto también me hizo confirmar que sí puedo sustentar el código: entiendo por qué cada estructura se usa y por qué el orden de verificación de meta (al sacar de la cola, no al insertar) es necesario para garantizar optimalidad.